ConectaCampus

Arquitetura distribuída

PJBL | Entrega 3 | 26 de outubro de 2026

ARC42, C4, Architecture Canvas e evidências

Implementação publicada no GitHub e implantada no Azure. Demonstrada em vídeo no YouTube, com a fala de todos os integrantes.

Validação	Estado
Testes locais e componentes	Aprovados
Interface desktop e móvel	Aprovada em três testes de navegador
Compilação React e Bicep	Aprovadas
Auditoria de dependências	Sem alertas
Integração real	Aprovada no GitHub Actions: MongoDB, SQL Server, Azurite, Functions Host, outbox e ledger
Cloud e Docker Hub	Publicados; smoke test aprovado no gateway Azure
Vídeo	Publicado no YouTube (não listado)

Equipe

Anna Quezia Dos Santos, Laura Rodrigues de Souza de Camargo, Leonardo Smijtink, Otávio Augusto Fermino da Silva

Projeto PJBL, Entrega 3. Implementação e preparação da demonstração.

1. Introdução e objetivos

O ConectaCampus reúne eventos acadêmicos e certificados. O objetivo técnico é demonstrar microfrontend React, BFF Node.js, dois microserviços, bancos distintos, gateway e Azure Functions. A apresentação deve usar URLs públicas e fala de todos os alunos.

Alunos: Anna Quezia Dos Santos, Laura Rodrigues de Souza de Camargo, Leonardo Smijtink e Otávio Augusto Fermino da Silva. Nomes preservados do GRUPO.md original.

2. Restrições

O frontend consome somente o BFF pelo gateway. Eventos pertencem ao MongoDB e certificados ao Azure SQL. Nenhum serviço acessa o banco do outro. Código público, READMEs, Swagger, Docker e testes são obrigatórios. A apresentação local não substitui a demonstração cloud.

A referência “Free 1 DTU” do enunciado difere da oferta gratuita atual do Azure SQL, baseada em cotas serverless. Confira a elegibilidade da assinatura. A implantação usa o tier Basic de 5 DTU, a opção mais próxima do "1 DTU" do enunciado: a oferta gratuita serverless teve a cota mensal esgotada em cerca de dois dias pela varredura constante da outbox. O custo é de poucos centavos por dia do crédito de estudante.

3. Contexto e escopo

Estudantes e organizadores consultam eventos e administram certificados. MongoDB Atlas, Azure SQL e Storage são dependências externas. Não há integração com sistemas acadêmicos institucionais ou autenticação por aluno nesta demonstração. Use dados fictícios.

Os modelos C4 de contexto, containers e componentes estão em docs/c4.md e nas páginas gráficas do PDF. O Architecture Canvas está em docs/canvas.md.

4. Estratégia da solução

A SPA possui módulos de Resumo, Agenda e Certificados, carregados separadamente. O Azure API Management centraliza a entrada na nuvem, e o Nginx cumpre esse papel apenas no Compose local. O BFF faz proxy dos CRUDs e agrega dados em GET /aggregated-data. Foram preservados os domínios do projeto original.

Clean Architecture direciona as dependências ao domínio. Na Application dos microserviços, cada operação CRUD tem seu slice. Infrastructure implementa repositórios, enquanto API expõe HTTP e Swagger.

5. Visão de blocos

O BFF consulta eventos e certificados em paralelo, calcula contagens e chama a Function HTTP. A resposta contém eventos, certificados, indicadores e generatedAt. O BFF não acessa bancos diretamente.

Eventos valida capacidade e normaliza lotação. Certificados valida carga horária de 1 a 1000 horas, data real e campos obrigatórios. Eventos usa ObjectId e certificados usa UUID. Drivers recebem campos permitidos e parâmetros.

A Function registra enrich e events por HTTP e campusEvents por Queue Trigger. O ledger no Table Storage usa id de evento como chave única. O contrato v1 não contém o nome do participante.

6. Visão de execução

Consulta: navegador, gateway, BFF, dois serviços e Function HTTP. Uma falha de dependência gera 502 ou 504. Um conjunto incompleto não é informado como sucesso completo.

Escrita: após validar, o SqlRepository grava certificado e outbox em uma transação. O publicador envia pendências e confirma publishedAt somente após envio. Uma falha entre envio e confirmação pode gerar reentrega.

Consumo: o Queue Trigger valida o contrato e cria a entidade no ledger. Chave existente significa duplicata. Mensagens inválidas vão para campus-events-poison após cinco falhas. O fluxo é eventualmente consistente e tem entrega pelo menos uma vez.

7. Visão de implantação

Compose contém gateway, frontend, BFF, serviços, MongoDB, SQL Server, Azurite e Azure Functions Host. Apenas a porta 8080 é publicada. O banco local e o schema SQL são inicializados. Credenciais ficam fora do Git.

Na nuvem (Azure, região Brazil South) o frontend roda no Azure Static Web Apps (https://happy-pebble-0ff30c50f.3.azurestaticapps.net). O Azure API Management, no tier Consumption, é o gateway público (https://apim-campus-7e6589.azure-api.net/api): roteia para o BFF, aplica CORS restrito ao site e injeta o segredo x-gateway-secret. BFF, Eventos e Certificados rodam em Container Apps, e o BFF só responde quando recebe o segredo. Azure SQL, Storage, Function App e MongoDB Atlas completam o ambiente. O Bicep em infra/ implanta os Container Apps e o API Management, e as imagens estão no Docker Hub (leosmkk/conectacampus-*:v1). Os cinco componentes ficam em pastas do repositório público único, cada um com Dockerfile próprio.

8. Conceitos transversais

Segurança: token interno entre serviços, Function key no servidor, segredo do gateway exigido pelo BFF, CORS restrito ao site, limites de payload e métodos no Nginx local e bancos sem portas publicadas no Compose. O API Management Consumption não oferece limite de requisições, o que é uma limitação conhecida. Não há login de usuário final. A demonstração deve conter somente dados de teste.

Observabilidade: X-Request-ID no BFF, logs de falha da outbox e id do evento processado. /events permite comparar publicação e ledger. Health informa processo ativo. Smoke verifica dependências e dados.

Testes: regras puras, CRUD HTTP, fronteiras de imports, interface em navegador e layout móvel. O pipeline de integração usa drivers e infraestrutura reais no Compose. A prévia local usa memória apenas de forma explícita.

9. Decisões de arquitetura

ADR 01: preservar eventos, certificados e a interface do ConectaCampus. ADR 02: manter bancos isolados por domínio. ADR 03: usar outbox para persistência atômica e consumidor idempotente para reentrega.

ADR 04: usar Azure Queue e Table Storage para mensageria e evidência de processamento. ADR 05: usar o Azure API Management (Consumption) como API Gateway na nuvem, mantendo tudo no Azure, e o Nginx apenas no Compose local. AWS API Gateway era a opção citada no enunciado. ADR 06: fixar mongo:7.0 no Compose, porque o mongo:8.0 não inicia em kernels Linux 6.19 ou mais novos (SERVER-121912). ADR 07: migrar o SQL do tier gratuito para o Basic e ampliar a varredura da outbox de 3 s para 30 s, porque a varredura curta mantinha o serverless ativo e consumiu os 100 mil vCore-segundos mensais.

10. Requisitos de qualidade

Correção: rejeitar capacidade impossível, carga horária inválida e data inexistente. Consistência: certificado e outbox confirmam ou revertem juntos. Resiliência: falha de fila mantém pendência e duplicatas não repetem o ledger.

Manutenibilidade: domínio puro e ausência de imports entre serviços. Operabilidade: smoke do fluxo completo. Usabilidade: estados de carregamento, erros e confirmação de exclusão, com textos diretos na interface.

11. Riscos e pendências

A implantação cloud e as imagens do Docker Hub estão publicadas, e o smoke test passou contra a URL do API Management. O vídeo da demonstração, com a fala dos quatro alunos, está publicado no YouTube como não listado: https://youtu.be/0Q4IWo1clfI.

A integração real roda no pipeline GitHub Actions (42405126580) com containers e bancos reais, e o mesmo smoke test passou contra o gateway Azure, cobrindo CRUD no MongoDB e no SQL, agregação, Function e mensageria. Prints locais e testes com memória não substituem a evidência cloud.

Polling da outbox e réplicas mínimas podem consumir recursos continuamente. O Azure SQL Basic fica sempre ligado e consome crédito de forma contínua (cerca de US$5 por mês), e os Container Apps mantêm uma réplica ativa. A outbox é varrida a cada 30 s (OUTBOX_POLL_MS), então um evento pode levar até 30 s para chegar à Function. Listagens têm limite de 1000 registros e ledger de 100 entidades. Indicadores cobrem esse conjunto. Paginação, autenticação institucional, migrações versionadas e rastreamento distribuído são evoluções.

12. Glossário e referências

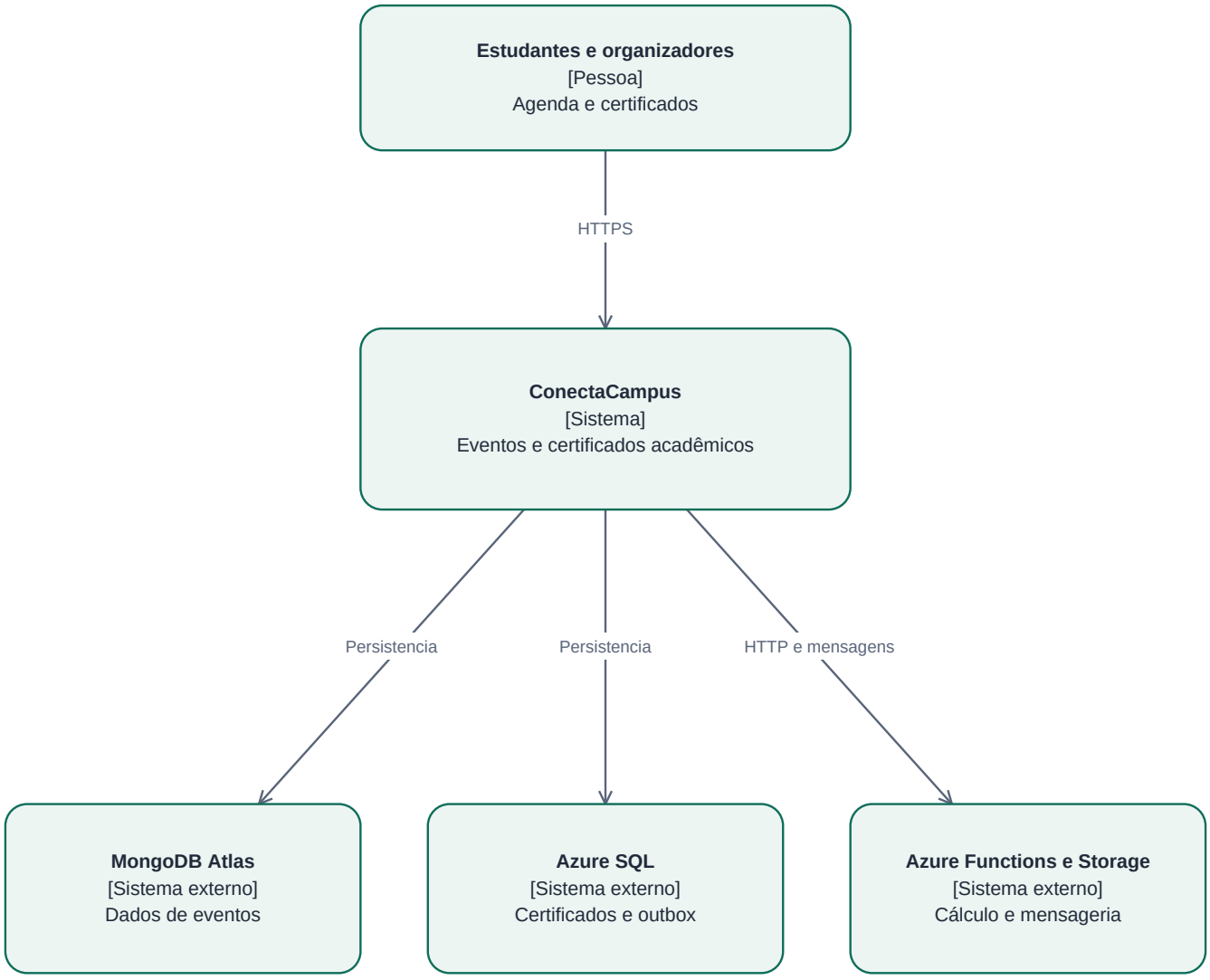
BFF: backend específico da interface. Slice: caso de uso por operação. Outbox: eventos na transação do dado de negócio. Ledger: registro idempotente. Gateway: entrada central com encaminhamento e controles.

Links verificados e pendências ficam em docs/links.json e na tabela do PDF. O vídeo da demonstração está em https://youtu.be/0Q4IWo1clfI (não listado). O roteiro do vídeo e o guia de publicação acompanham o projeto.

Referências oficiais: https://learn.microsoft.com/azure/azure-functions/functions-reference-node e https://learn.microsoft.com/azure/azure-functions/functions-bindings-storage-queue-trigger. Oferta SQL: https://learn.microsoft.com/azure/azure-sql/database/free-offer. Ingress: https://learn.microsoft.com/azure/container-apps/ingress.

C4: contexto

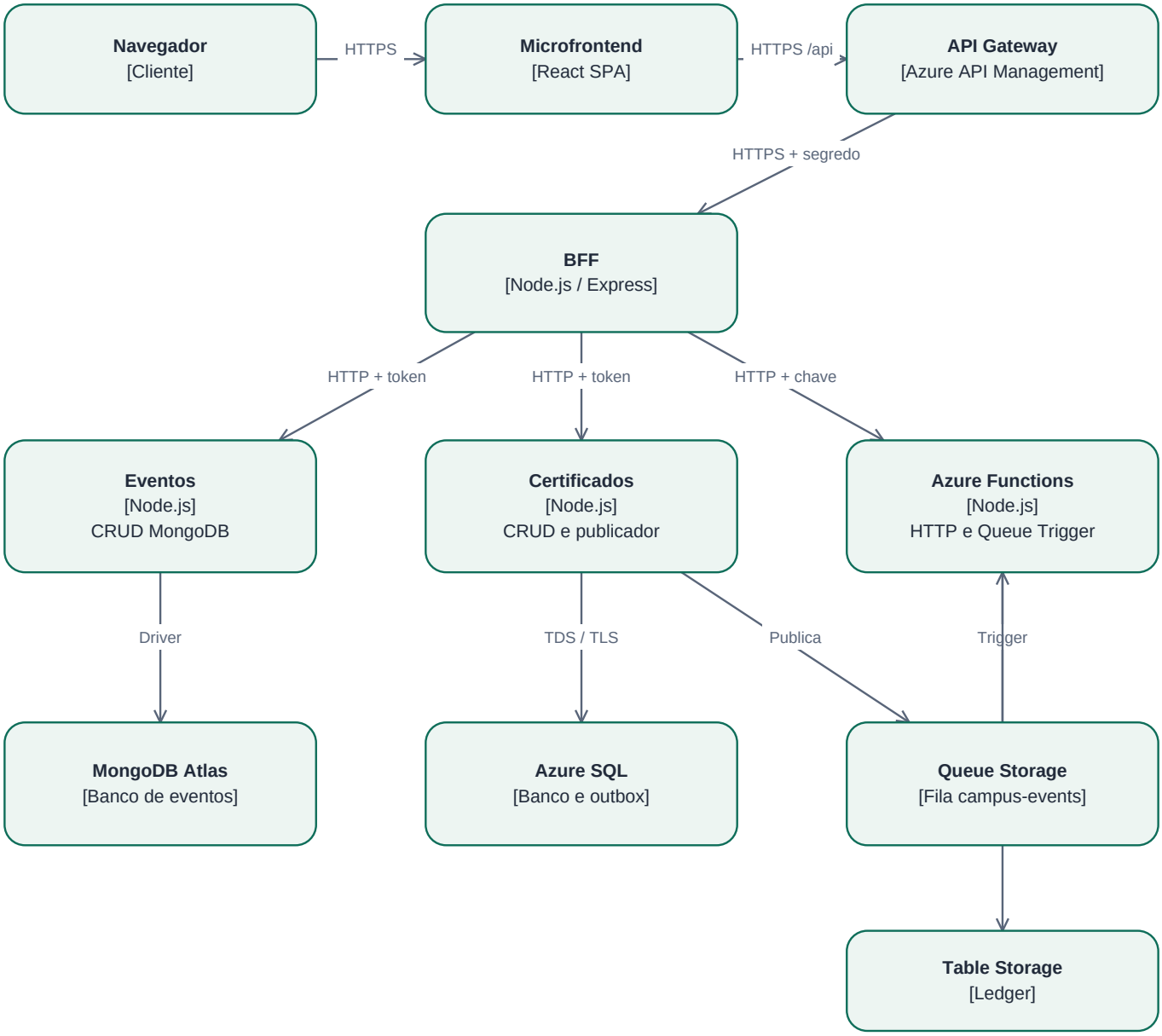
Nível 1. Pessoas, sistema e dependências externas.



Escopo funcional: eventos e certificados. A infraestrutura Azure está implantada e validada por smoke test.

C4: containers

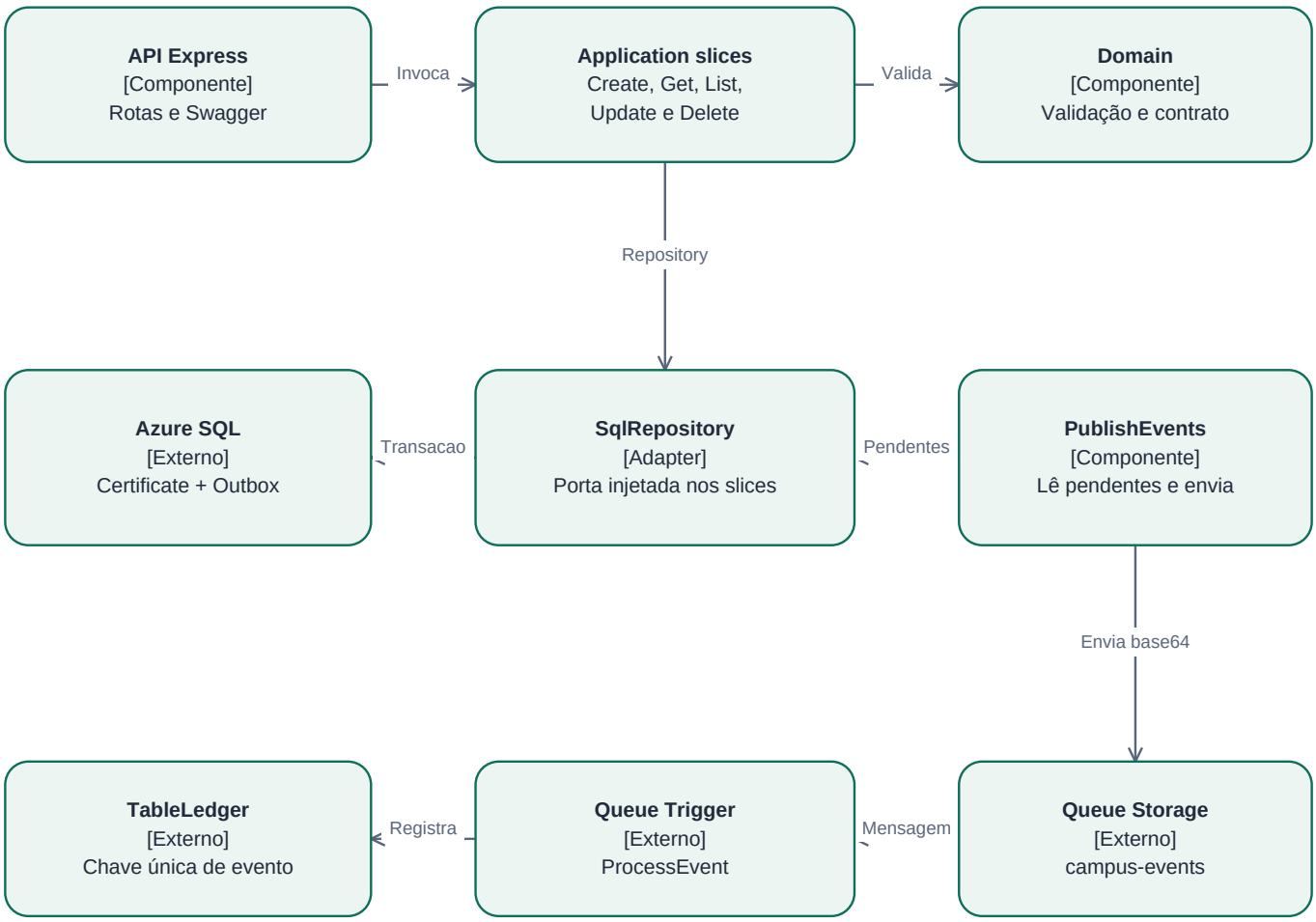
Nível 2. Chamadas síncronas e processamento assíncrono.



O código React é servido pelo Static Web Apps e chama /api no API Management. O BFF só responde com o segredo que o gateway injeta. A Function HTTP usa chave de acesso.

C4: componentes

Nível 3. Interior do microserviço de certificados.



O publicador confirma publishedAt depois do envio. Uma falha intermediária gera reentrega. O consumidor trata conflito de chave como duplicata. A consistência é eventual.

Architecture Canvas

Área	Decisão
Problema	Centralizar agenda acadêmica e certificados de participação
Pessoas	Estudantes, organizadores acadêmicos e avaliadores do PJBL
Valor	Consultar vagas e administrar registros de eventos e certificados
Domínios	Eventos no MongoDB, certificados no SQL
Entradas	React pelo gateway, BFF HTTP e trigger de fila
Integrações	MongoDB Atlas, Azure SQL, Functions, Queue e Table Storage
Qualidade	Invariantes no domínio, dependências voltadas para dentro, transação e idempotência
Implantação	Imagens no Docker Hub, Azure API Management como gateway, containers no Container Apps, frontend no Static Web Apps, Function protegida por chave
Riscos	Cotas de nuvem, cold start, segredos, eventual consistência e mensagens na poison queue
Evidências	Testes locais, pipeline de integração e demonstração final em URLs públicas
Pendências	Nenhuma para a entrega: vídeo publicado no YouTube (não listado)

Links e estado da entrega

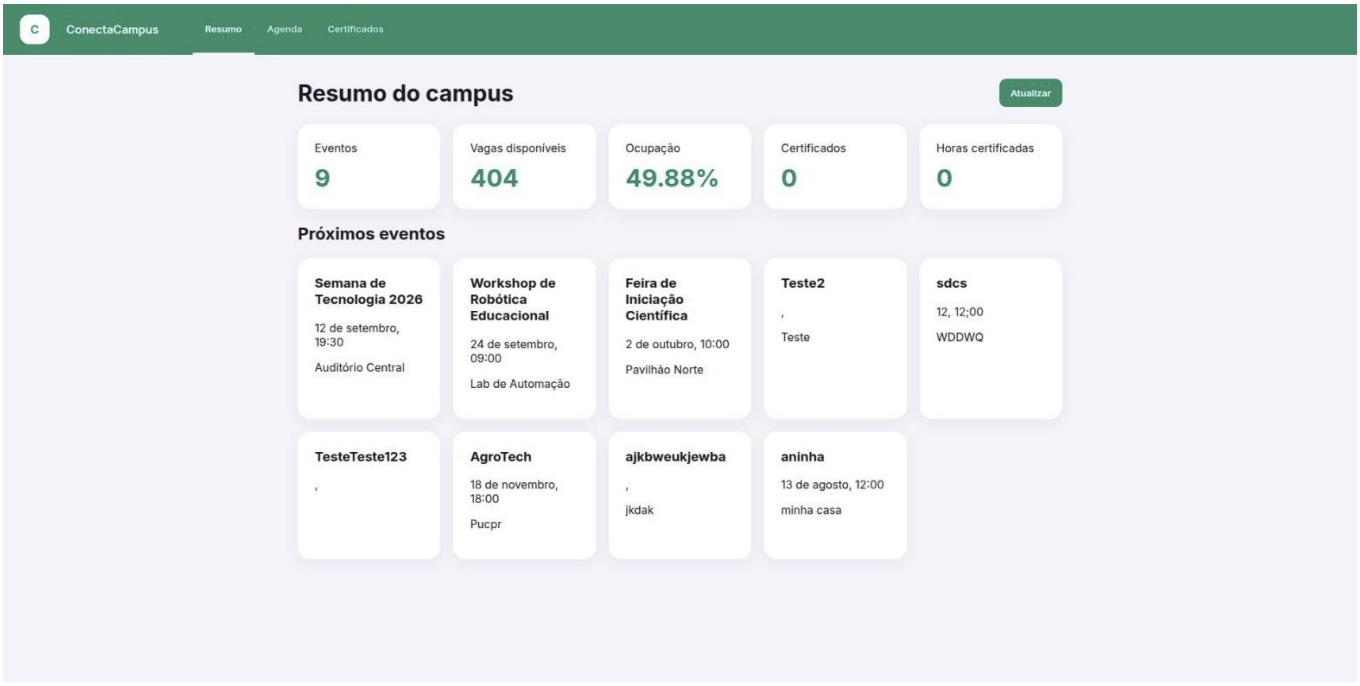
Destino	Link ou estado
microfrontend	https://github.com/leosmkk/conectacampus/tree/master/frontend
bff	https://github.com/leosmkk/conectacampus/tree/master/bff
mongodb	https://github.com/leosmkk/conectacampus/tree/master/services/eventos
sql	https://github.com/leosmkk/conectacampus/tree/master/services/certificados
function	https://github.com/leosmkk/conectacampus/tree/master/azure-function
Integração	https://github.com/leosmkk/conectacampus
Pipeline	https://github.com/leosmkk/conectacampus/actions/runs/42405126580
dockerHub	https://hub.docker.com/u/leosmkk (leosmkk/conectacampus-bff, -eventos, -certificados, -frontend, -function, -gateway : v1)
frontend	https://happy-pebble-0ff30c50f.3.azurestaticapps.net
gateway	https://apim-campus-7e6589.azure-api.net/api
swaggerBff	https://apim-campus-7e6589.azure-api.net/api/docs/
swaggerMongo	https://apim-campus-7e6589.azure-api.net/api/docs/?url=/api/specs/eventos
swaggerSqlAndEvents	https://apim-campus-7e6589.azure-api.net/api/docs/?url=/api/specs/certificados
youtube	https://youtu.be/0Q4IWo1clfI

Caminhos após a publicação

No gateway público: /api/docs/ para BFF, /api/docs/?url=/api/specs/eventos para Mongo e /api/docs/?url=/api/specs/certificados para SQL e eventos. O contrato assíncrono está em docs/asyncapi.yaml.

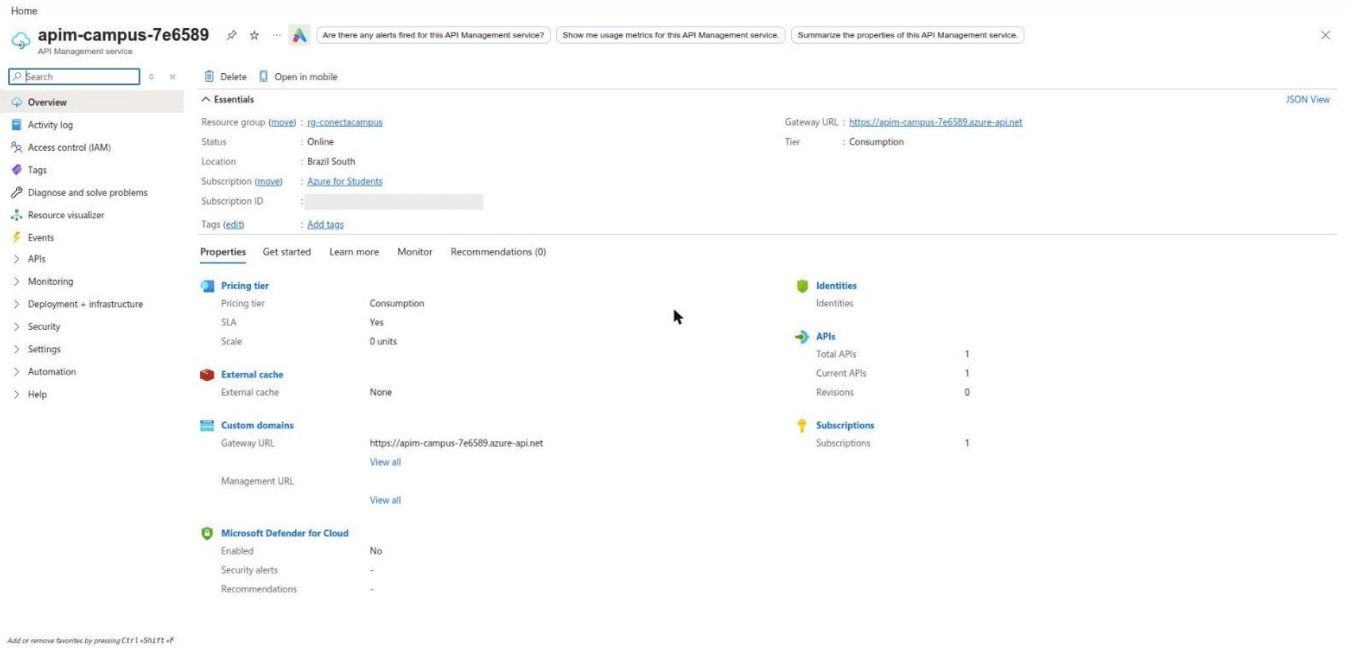
Frontend em funcionamento (nuvem)

Azure Static Web Apps (happy-pebble-0ff30c50f.3.azurestaticapps.net). O resumo vem de GET /api/aggregated-data no API Management.



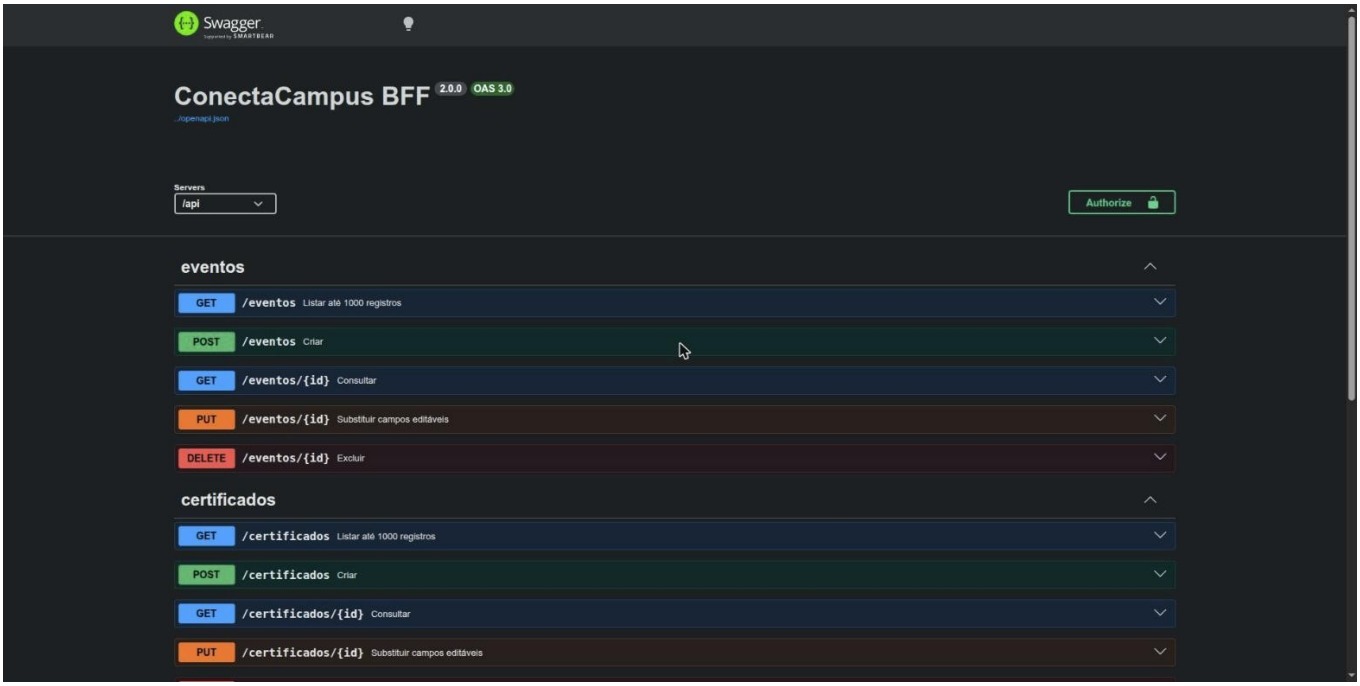
API Gateway no Azure (API Management)

Portal Azure: serviço apim-campus-7e6589, tier Consumption, Brazil South, 1 API. ID da assinatura e dados da conta ocultados.



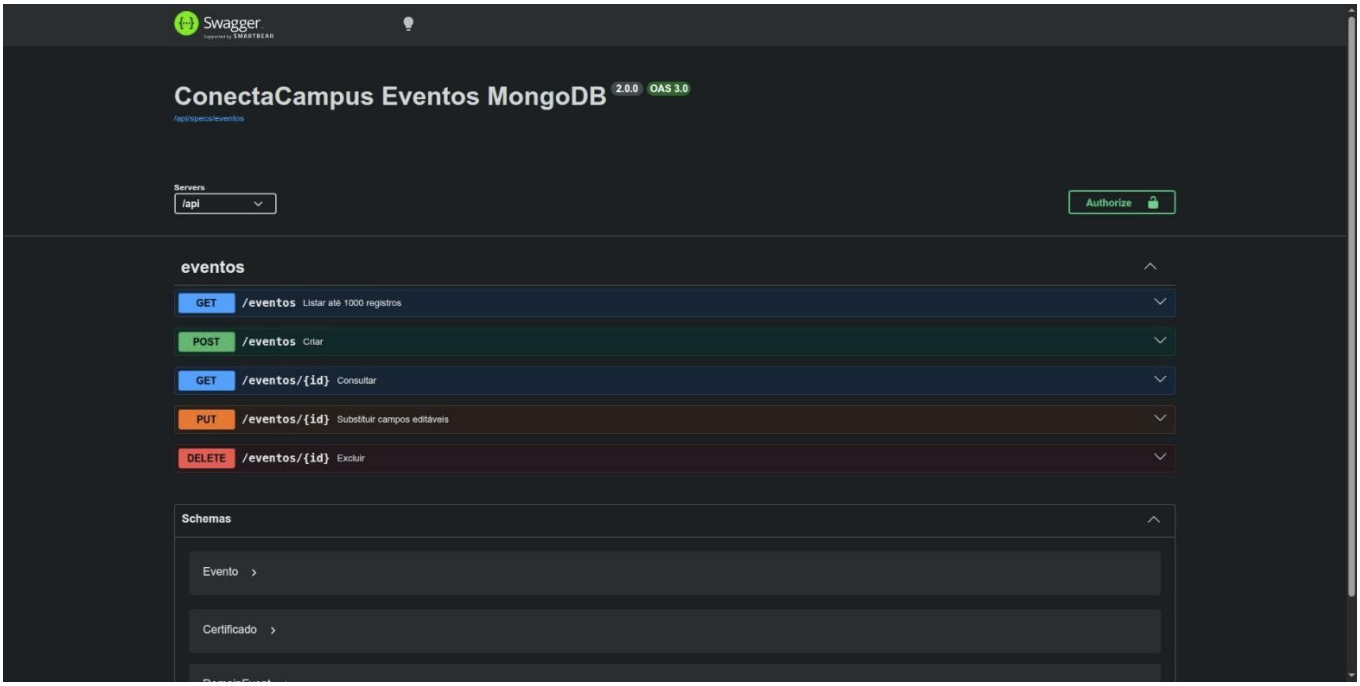
Swagger BFF (nuvem)

https://apim-campus-7e6589.azure-api.net/api/docs/



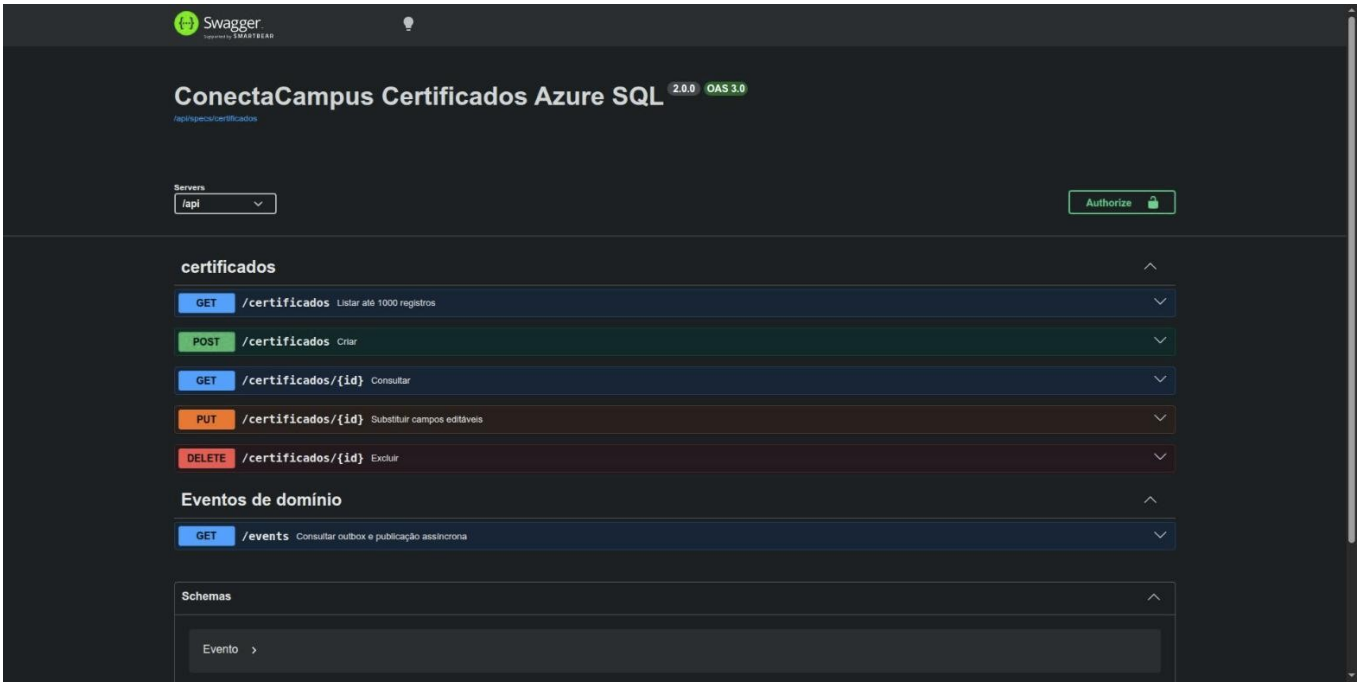
Swagger microserviço MongoDB (nuvem)

https://apim-campus-7e6589.azure-api.net/api/docs/?url=/api/specs/eventos



Swagger microserviço SQL e eventos (nuvem)

https://apim-campus-7e6589.azure-api.net/api/docs/?url=/api/specs/certificados



Vídeo e conclusão da entrega

Vídeo publicado no YouTube (não listado): https://youtu.be/0Q4IWo1clfI. Mostra URLs de nuvem e a fala dos integrantes.

Vídeo publicado no YouTube como não listado: https://youtu.be/0Q4IWo1clfI. Cada integrante gravou a própria fala, e o vídeo mostra URLs públicas durante toda a demonstração.

1. Anna Quezia Dos Santos: apresentar o objetivo, abrir o frontend na URL cloud e mostrar Resumo e Agenda.

2. Laura Rodrigues de Souza de Camargo: criar, consultar, editar e excluir um evento. Mostrar validação de vagas e Swagger do microserviço Mongo pelo gateway.

3. Leonardo Smijtink: criar, consultar, editar e excluir um certificado. Demonstrar emissão, SQL e a outbox na mesma transação.

4. Otávio Augusto Fermino da Silva: mostrar GET /aggregated-data, os dois serviços, a chamada da Function e os eventos processados. Apresentar gateway, C4, imagens Docker Hub e testes.

Antes de gravar, rodar o smoke contra a URL pública. Durante a explicação, mostrar a aba de rede do navegador com as chamadas ao gateway/BFF, os registros de dados nos dois bancos e o processamento na fila. Não mostrar segredos nos painéis ou no terminal.

Evidência do vídeo

Link do vídeo: https://youtu.be/0Q4IWo1clfI